

Chapter 4 CPU Design

Tips

These instructions have only one step
DEC, INC, COM, JREL, JMP, SKIP

These instructions have two steps
AND, OR, ADD, SUB, LDA, NAND, JTR, XOR
Leg 193

⊗ Note: If DEC2 or INC2 then perform ~~two~~ decrement or increment twice.

Register Section

→ All the register section figure is same.
Except, if question has JREL then just make parallel adder in front of PC.

ALU Section

→ The operations performed in Accumulator are shown in ALU.

ADD, SUB, AND, OR, NOT, COM

Note: INC and DEC are done in accumulator
SKIP is done in PC ✓

No need to worry,
given in question

If Subtract ; SUB1: ~~DR~~ ← A DR ← M
 SUB2: AC ← AC + DR + 1

If LOA { M[AAAAAA] ← AC }
 ↳ LOA1: DR ← AC
 LOA2: M ← DR

Hardwired Control unit

00, 01, 10, 11 1ai 1 - - 0 ma rather,

it will be

1000
1010
1100
1110

→ Generated control signals are taken from registers.

Horizontal microcode

Vertical microcode

(30) (23 marks)

Date: _____

Page: _____

→ Pg: 120

i)	Instruction	Instruction Code	Operation
	ADD	00AAAAAA	$AC \leftarrow AC + M[AAAAAA]$
	AND	01AAAAAA	$AC \leftarrow AC \wedge M[AAAAAA]$
	JMP	10AAAAAA	GOTO AAAAAA
	INC	11XXXXXX	$AC \leftarrow AC + 1$

- Design the CPU's register section state diagram and ALU
- Design the hardwired control unit
- Design micro-sequencer control unit

i) State diagram, register and ALU
Ans: Here,

~~For~~ For a very simple CPU the registers involved are,

8-bit Accumulator (AC) register, 8-bit data register (DR), 6-bit Program Counter (PC), 6-bit address register (AR) and 2-bit instruction Register (IR).

Now,

Fetch 1: $AR \leftarrow PC$

Fetch 2: $DR \leftarrow M, PC \leftarrow PC + 1$

Fetch 3: $AR \leftarrow DR[5..0], IR \leftarrow DR[7..6]$

Now, ~~the~~ the RTL code of each instruction are as follow:

ADD:

ADD1: $DR \leftarrow M$

ADD2: $AC \leftarrow AC + DR$

AND:

AND1 : $DR \leftarrow M$

AND2 : $AC \leftarrow AC \wedge DR$

IMP

IMP1 : $PC \leftarrow DR[5..0]$

INC

INC1 : $AC \leftarrow AC + 1$

The state diagram is,

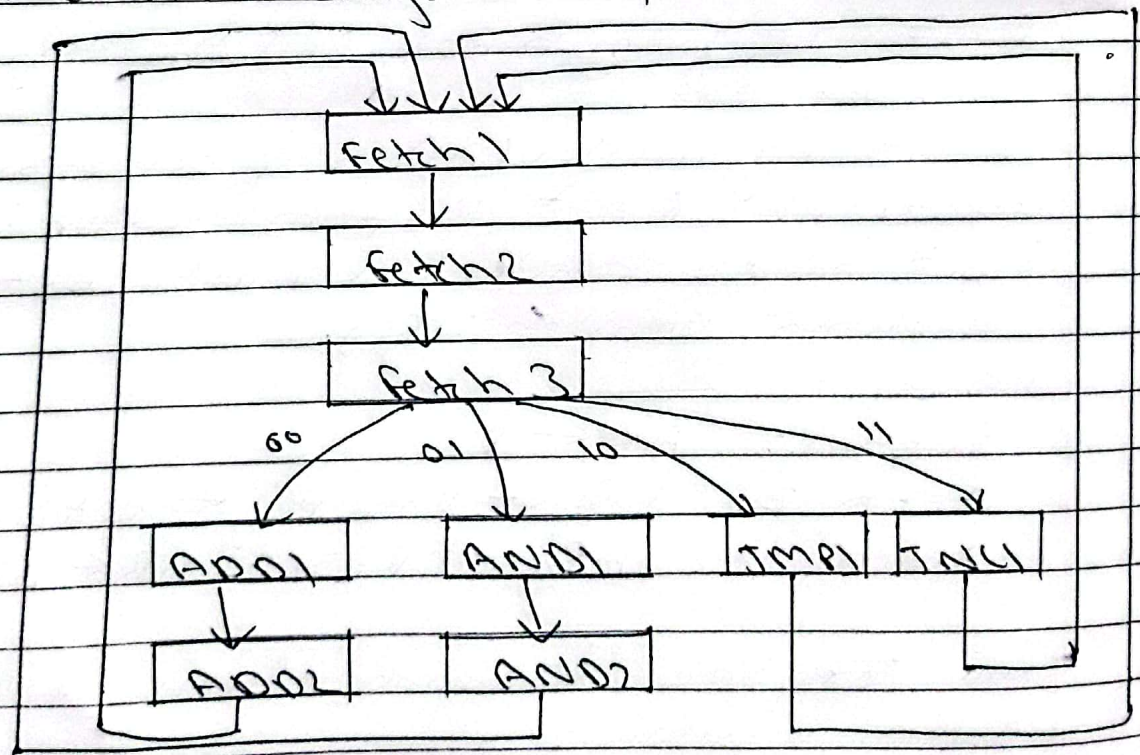
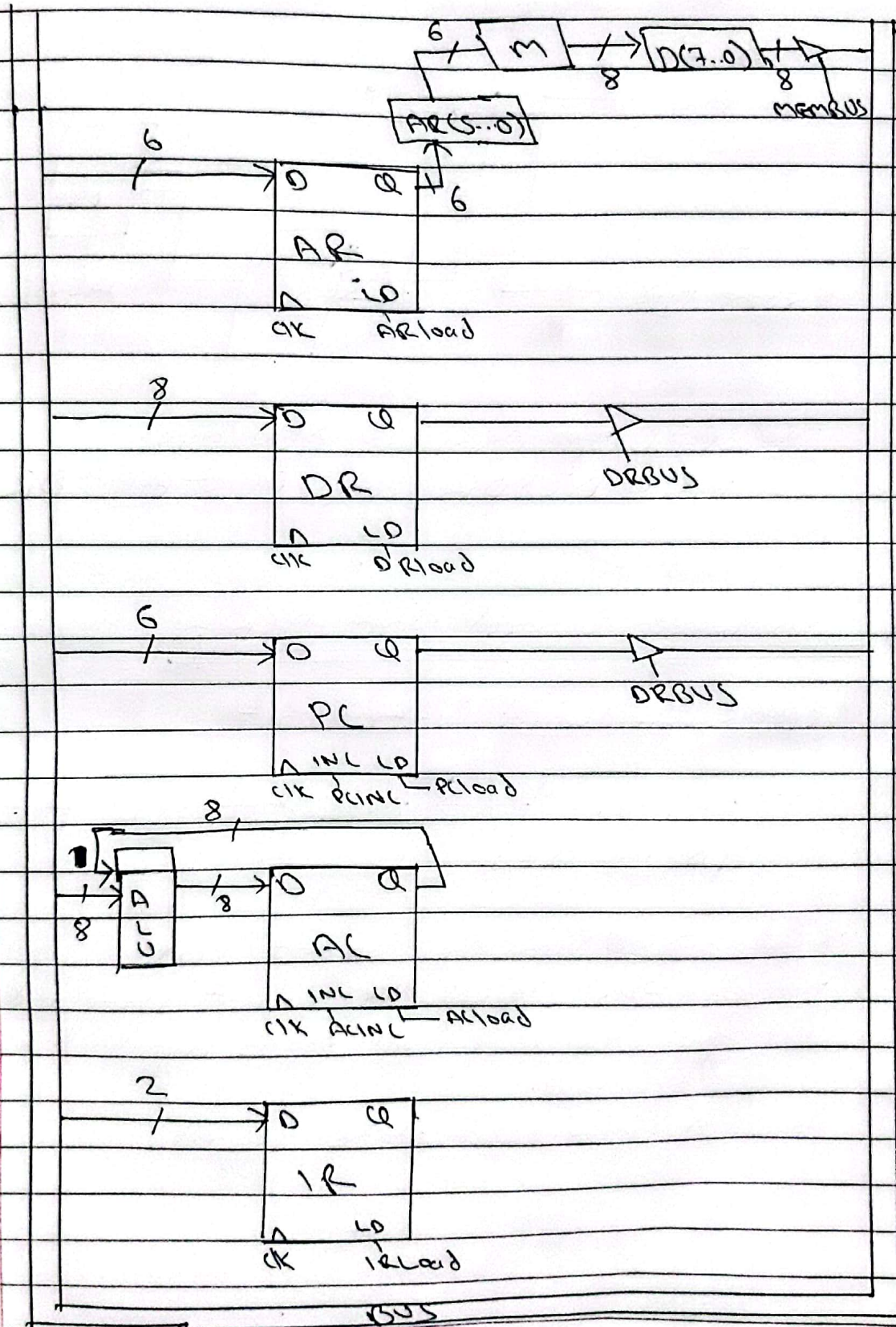


Fig. Final state Diagram

Register section for a very simple CPU by Bus Connection

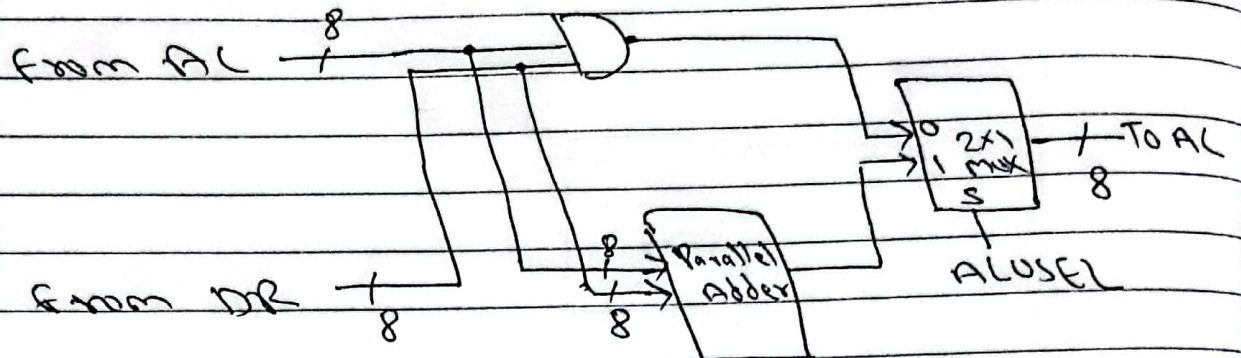


T.P.s: Make ~~Fig~~ large figure

Date: _____

Page: _____

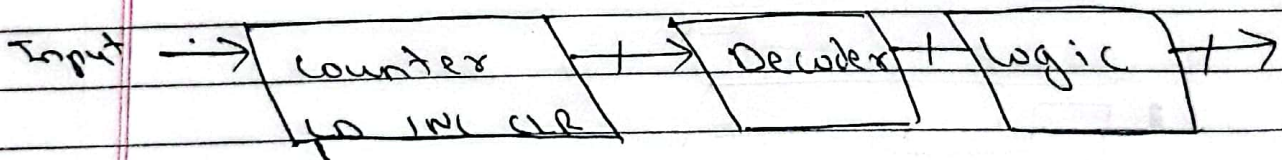
ALU Section for very simple CPU
⇒ ALU Design:



The RTL operations in ALU are
 $AC \leftarrow AC \vee DR$ and $AC \leftarrow AC \wedge DR$

ii) Hardwired Control Unit Design

Generic Hardwired Control Unit



Counter → contains the current state.

Decoder → it takes the current state and generates individual ~~state~~ signals for each state.

Logic → individual state signals

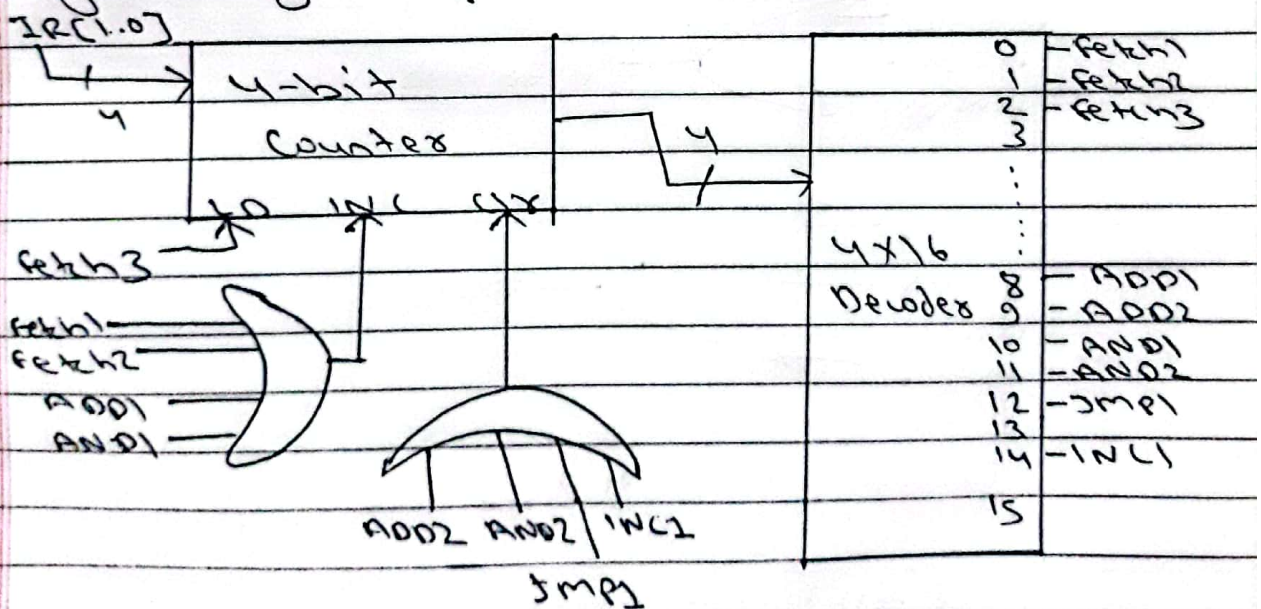
Now, for given CPU there are total 9 states.

Since there are 9 states, 4×16 decoder is required and a 4 bit counter is required.

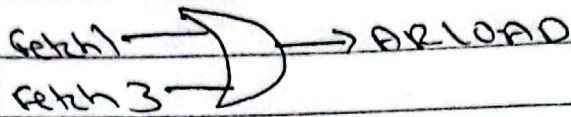
Creating a table

Instruction	1st state	IR	Counter Value
ADD	ADD1	00	1000 (8)
AND	AND1	01	1010 (10)
JMP	JMP1	10	1100 (12)
INC	INC1	11	1110 (14)

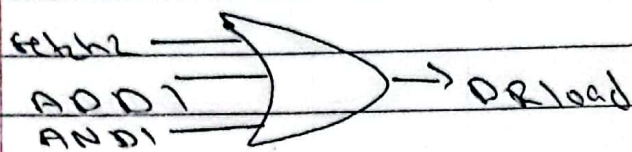
Now, the hardwired control unit design for a given very simple CPU is obtained as follows:



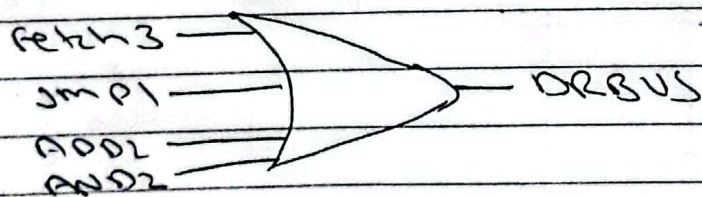
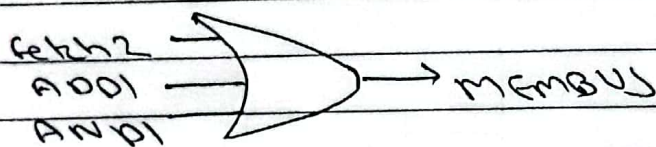
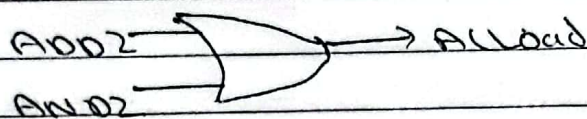
Generating Control Signals



jmp1 → PLLoad



fech3 → IRload



fech1 → PLBUS

ADD2 → ALUSEL

INC1 → ACINC

mainly sometimes horizontal is only asked,
sometimes vertical is asked.
If not mentioned, make horizontal microcode

Date: _____

Page: _____

iii) Micro-Sequencer Control Unit Design

→ There are two types → ~~horizontal~~ horizontal microcode
→ vertical microcode
{only one will be asked}

→ Horizontal microcode

→ Assign each state a address in the microcode.
Refer decoder of hardwired control unit.

State	Address
Fetch 1	0000
Fetch 2	0001
Fetch 3	0010
ADD1	1000
ADD2	1001
AND1	1010
AND2	1011
IMPL	1100
INCL	1110

Now, assign the mnemonics to each microoperations available; then these mnemonics represent the name of the microoperation signals in the microcode memory.

→ ~~Perform~~ the given vertical microcode, you need to follow upto this step.

Date: _____

Page: _____

mnemonics

microoperations

ARPC

$AR \leftarrow PC$

DRM

$DR \leftarrow M$

PCIN

$PC \leftarrow PC + 1$

ARDR

$AR \leftarrow DR$

IRDR

$IR \leftarrow DR$

ACPLUS

$AC \leftarrow AC + DR$

ACAND

$AC \leftarrow AC \wedge DR$

PCDR

$PC \leftarrow DR$

ACIN

$AC \leftarrow AC + 1$

Now, the microcode value of the ARDR and IRDR will be same for all the states, we replace them with a single ARDR. So,

	State	Address	SEL	ARPC	DRM	ARDR	ACPLUS	ACAND	PCIN	PCDR	ACIN	ADDR
	Fetch1	0000	0	1	0	0	0	0	0	0	0	0001
	Fetch2	0001	0	0	1	0	0	0	1	0	0	0010
	Fetch3	0010	1	0	0	1	0	0	0	0	0	XXXX
	ADD1	1000	0	0	1	0	0	0	0	0	0	1001
	ADD2	1001	0	0	0	0	1	0	0	0	0	000
1	AND1	1010	0	0	1	0	0	0	0	0	0	1010
1	AND2	1011	0	0	0	0	0	1	0	0	0	0000
	JMP1	1100	0	0	0	0	0	0	0	1	0	0000
	INC1	1110	0	0	0	0	0	0	0	0	1	0000

Now, using the micro-operations signals to generate control signals required by the rest of the CPU.

Signal	Value
ADRLoad	DRM
ARload	ARPC $\vee$ AIDR
ALload	ALPLUS $\vee$ ALAND
PCload	PCDR
IRload	AIDR
MEMBUS	DRM
DRBUS	AIDR $\vee$ PCDR $\vee$ ALPLUS $\vee$ ALAND
PCBUS	ARPC
ALINC	ALIN
PCINC	PCIN
ALUSFL	ALAND

→ Create memories is same like in horizontal

Date: _____

Page: _____

Generating micro-operations using vertical microcode.

- In vertical microcode, the micro-operations are grouped into fields.
- Here, DRM and PCIN occur during the same state (Febh 2). So, assign them to different fields.
- We need minimum two fields.
- Let them assign name M₁ & M₂.

M ₁		M ₂	
Value	micro-operation	Value	micro-operation
000	NOP	0	NOP
001	PCIN	1	DRM
010	PCDR		
011	AIDR		
100	ARPC		
101	ACIN		
110	ACPLUS		
111	ACAND		

Now, using these information, generation of the final vertical microcode for the given very simple CPU as:

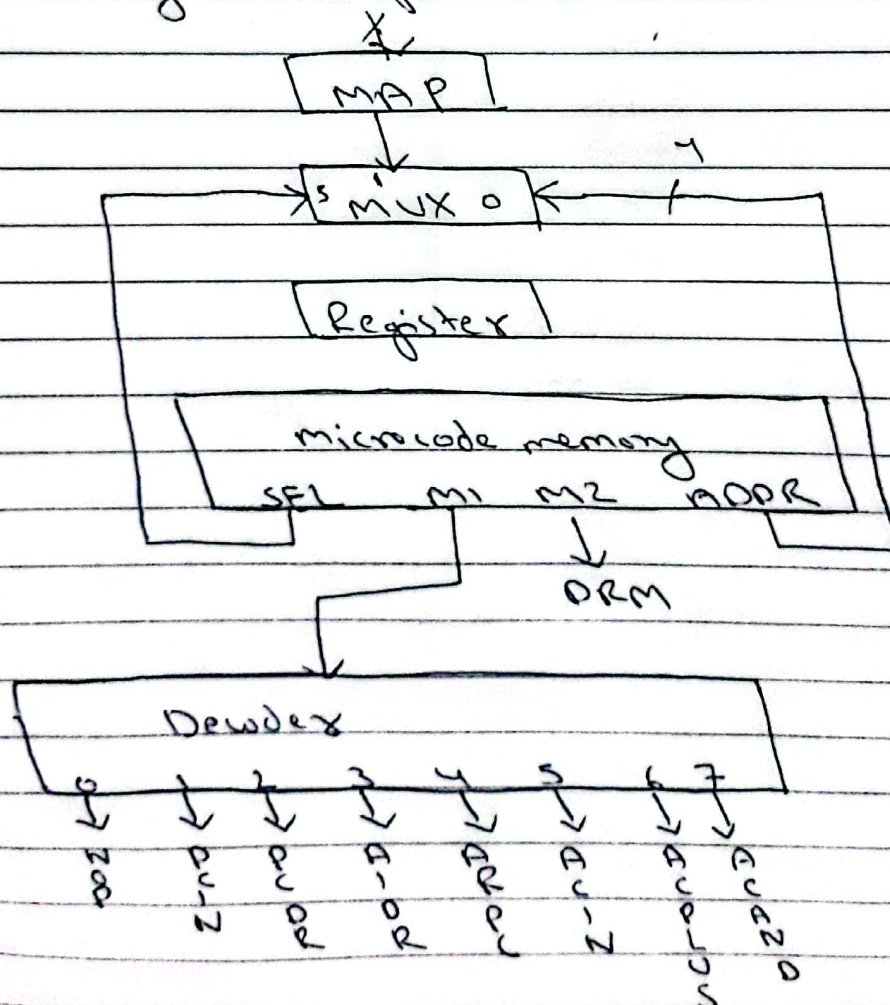
→ need to refer previous parts

Date: _____

Page: _____

State	Address	SEL	M1	M2	ADDR
Fetch 1	0000	0	100	0	0001
Fetch 2	0001	0	001	1	0010
Fetch 3	0010	1	011	0	XXXX
ADD1	1000	0	000	1	1001
ADD2	1001	0	110	0	0000
AND1	1010	0	000	1	1011
AND2	1011	0	111	0	0000
JMP1	1100	0	010	0	0000
JMP2	1110	0	101	0	0000

Generating micro-operations from vertical microcode:



2. A CPU with the same registers as very simple CPU, has following instructions set.

Instruction	Instruction Code	Operation
COM	00XXXXXX	$AC \leftarrow AC'$
JREL	01000000	$PC \leftarrow PC + 00000000$
OR	10000000	$AC \leftarrow AC \vee M[00000000]$
SUB1	11000000	$AC \leftarrow AC - M[00000000] - 1$

i) Show the state diagram, register section and ALU.

ii) Design hardwired control unit and control signal generators.

iii) Design a micro-sequencer code with unit with horizontal microcode and show the micro-operations generation.

i)

→ For a very simple CPU the registers involved are: PC (6 bit), AR (6 bit), DR (8 bit), AC (8 bit), IR (12 bit)

Now, the operations are

Fetch 1: $AR \leftarrow PC$

Fetch 2: $DR \leftarrow M, PC \rightarrow PC + 1$

Fetch 3: $IR \leftarrow DR[7..0], AR \leftarrow DR[5..0]$

COM1: $AC \leftarrow AC'$

JREL1: $PC \leftarrow PC + DR$

OR 1: $AC \leftarrow AC \vee DR$

OR 2: $AC \leftarrow AC \vee DR$

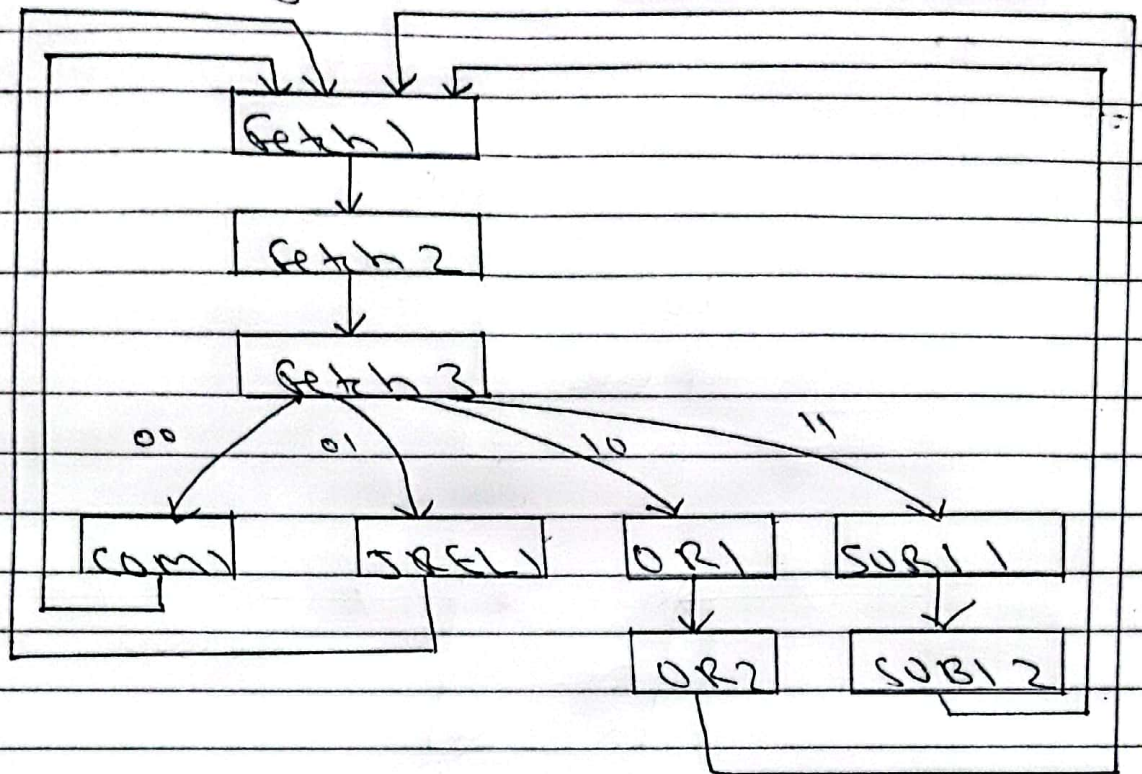
SUB11: $DR \leftarrow M$

SUB12: $AC \leftarrow AC - DR - 1$

$AC \leftarrow AC + \overline{DR} + 1 - 1$

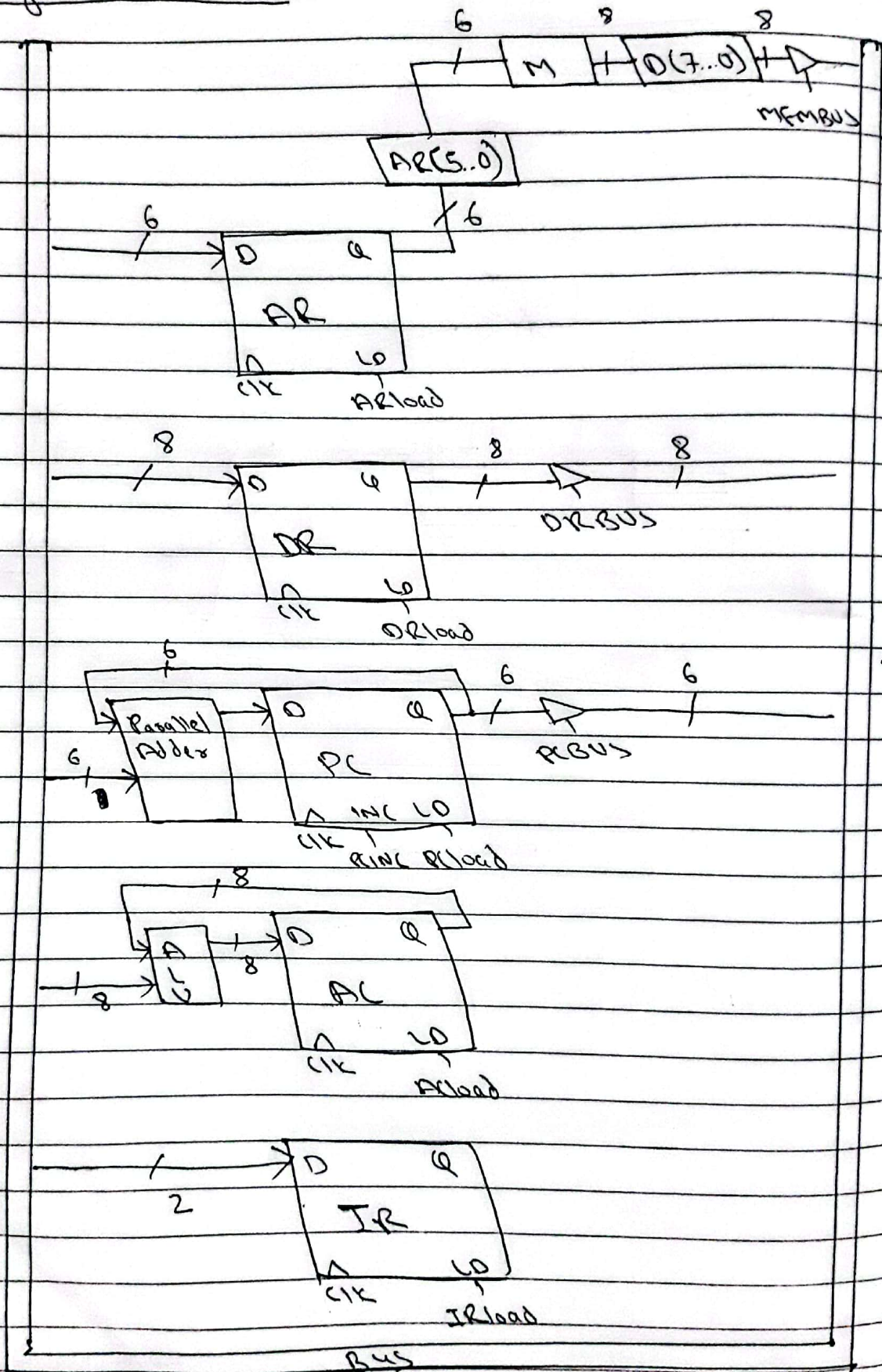
$AC \leftarrow AC + \overline{DR}$

State Diagram



P.T.O →

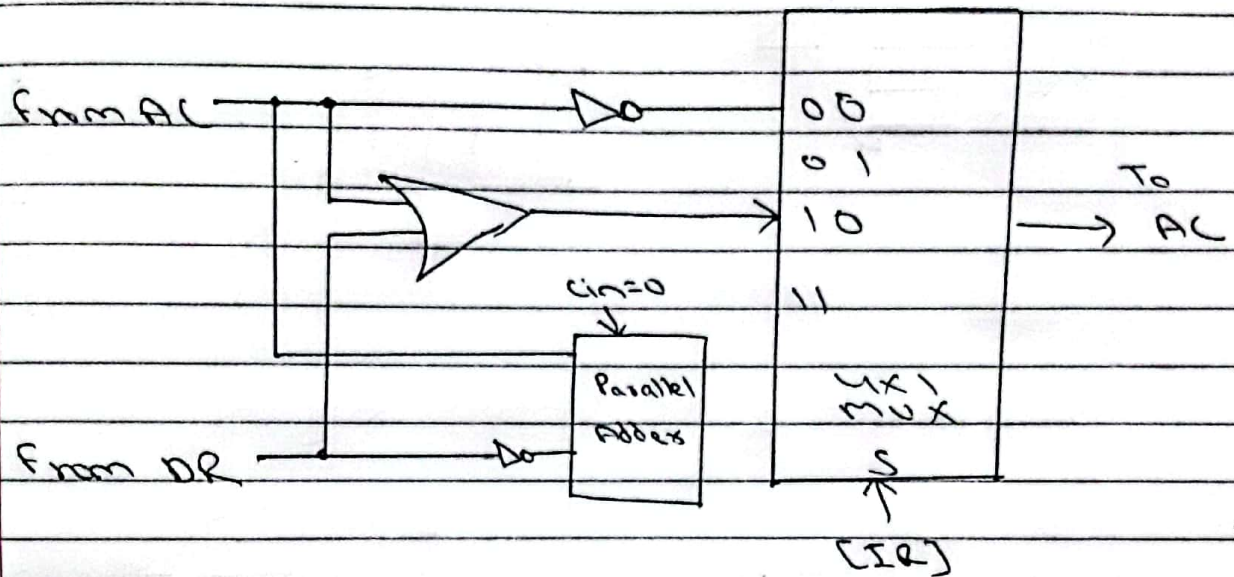
Register Section



ALU Section

→ The micro-operations that occur in ALU are:

$$AC \leftarrow AC', AC \leftarrow AC \vee DR, AC \leftarrow AC \wedge DR$$

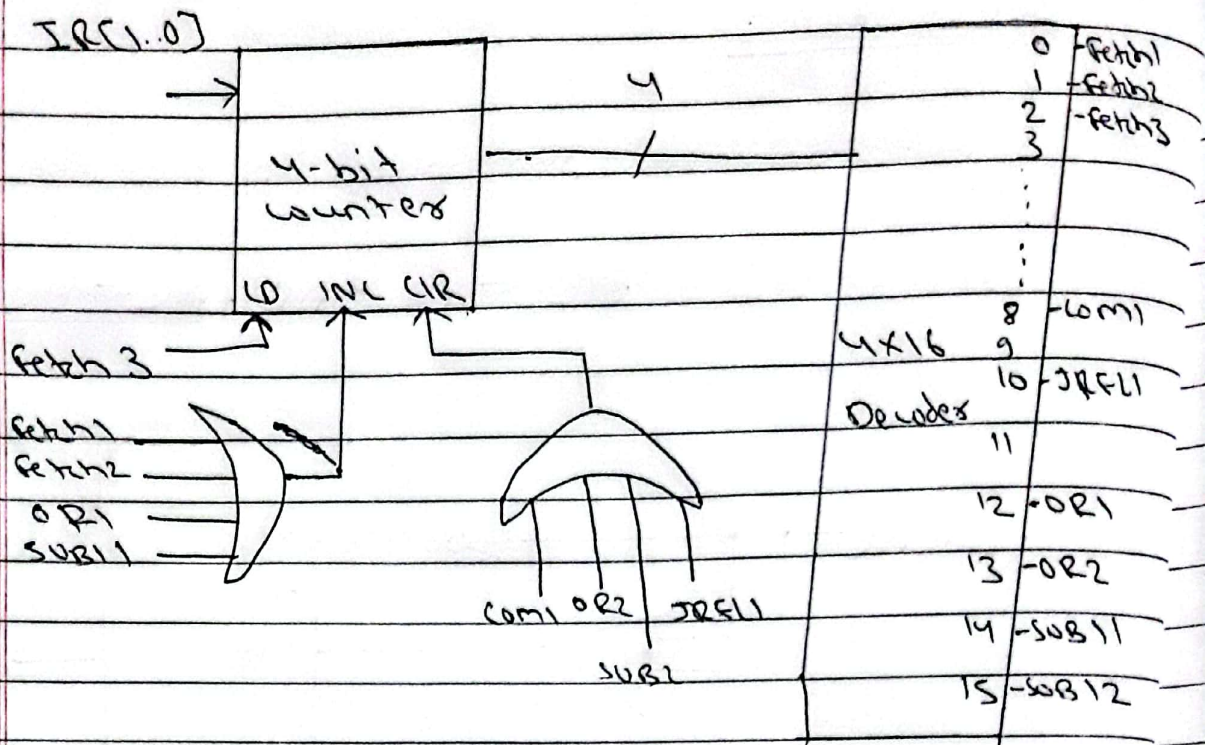


ii) Hard-wired control Unit

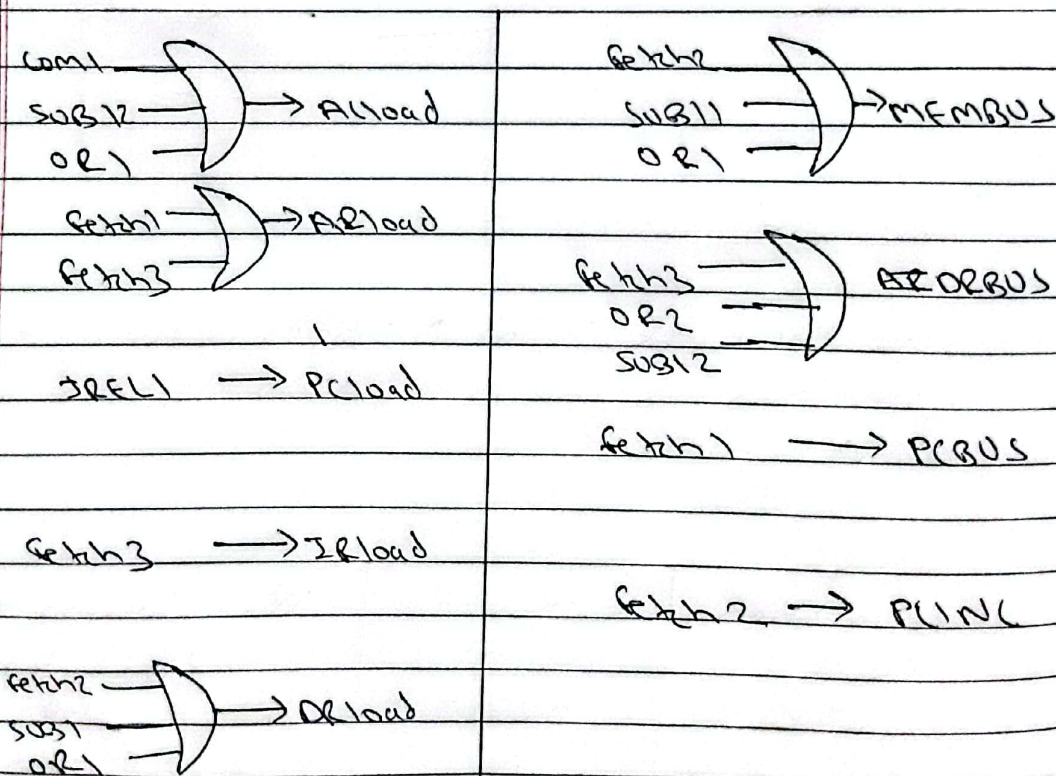
→ Since, there are total 9 states in the given instruction set, we need 4x16 decoder and 4-bit counter.

Table 1

Instruction	First State	IR	Counter Value
COM	COM1	00	1000 (8)
JREL	JREL1	01	1010 (10)
OR	OR1	10	1100 (12)
SUB1	SUB11	11	1110 (14)



Now, generating the control signals



iii) Microsequences Control Unit Design

a) Horizontal microcode

Assigning address to each state

State	Address
fetch1	0000
fetch2	0001
fetch3	0010
COM1	1000
JREL1	1001 1010
OR1	1100
OR2	1101
SUB1	1110
SUB2	1111

Now,

assigning mnemonics to each micro-operation available; then these mnemonics represents the name of the micro-operations signals in the microcode memory.

Mnemonics	Microoperations
ARPL	$AR \leftarrow PC$
DRM	$DR \leftarrow M$
PLIN	$PC \leftarrow PC + 1$
ARDR	$AR \leftarrow DR$
IRDR	$IR \leftarrow DR$
ACCOM	$AC \leftarrow AC'$
PCPLUS	$PC \leftarrow PC + DR$
ACOR	$AC \leftarrow AC \vee DR$
ACPLUS	$AC \leftarrow AC + DR$

Now, since the microcode value of ARDR and IRDR will be same for all the states, so we replace them with a single AIDR

State	Address	S L	A R C	I R M	P C N	A I R	A I R	A I R	A I R	A I R	A I R	ARDR
Fetch1	0000	0	1	0	0	0	0	0	0	0	0	0001
Fetch2	0001	0	0	1	1	0	0	0	0	0	0	0010
Fetch3	0010	1	0	0	0	1	0	0	0	0	0	XXXX
COM1	1000	0	0	0	0	0	1	0	0	0	0	0000
REF1	1010	0	0	0	0	0	0	1	0	0	0	0000
OR1	1100	0	0	1	0	0	0	0	0	0	0	1101
OR2	1101	0	0	0	0	0	0	0	1	0	0	0000
SUB1	1110	0	0	1	0	0	0	0	0	0	0	1111
SUB2	1111	0	0	0	0	0	0	0	0	1	0	0000

Now generating the control signals needed by the rest of the CPU

Control signal		Value
ALL		
Refer to Hardware Control Signals	ARload	ARPL $\vee$ AIDR
	PCload	PCPLUS
	DRload	DRM
	ALoad	ALCOM $\vee$ PCPLUS $\vee$ DRM
	IRload	AIDR
	PCINC	PCIN
	MEMBUS	DRM
	DRBUS	AIDR $\vee$ AIDR $\vee$ AIBUS
	PCBUS	ARPL

→ Question asked only horizontal but doing this for practice. Date: _____ Page: _____

b) Vertical micro-code

- Here, we need to group the microoperations into different fields. for that we follow the rules.
- If the two microoperations occur during the same state, assign them to different fields.
- In this case, DRM and PCIN occur during fetch 2. So assign them to different fields. (let M_1 and M_2 be two fields)
- Microoperations modifying the same registers should be placed together in a same field.

M_1		M_2	
values	Microoperation	values	Microoperation
000	NOP	0	NOP
001	PCIN	1	DRM
010	PCPLUS		
011	ARPL		
100	ACOR		
101	ACPLUS		
110	ACOM		
111	AIDR		

Refer RTL steps to know which
operation occurs at which steps.

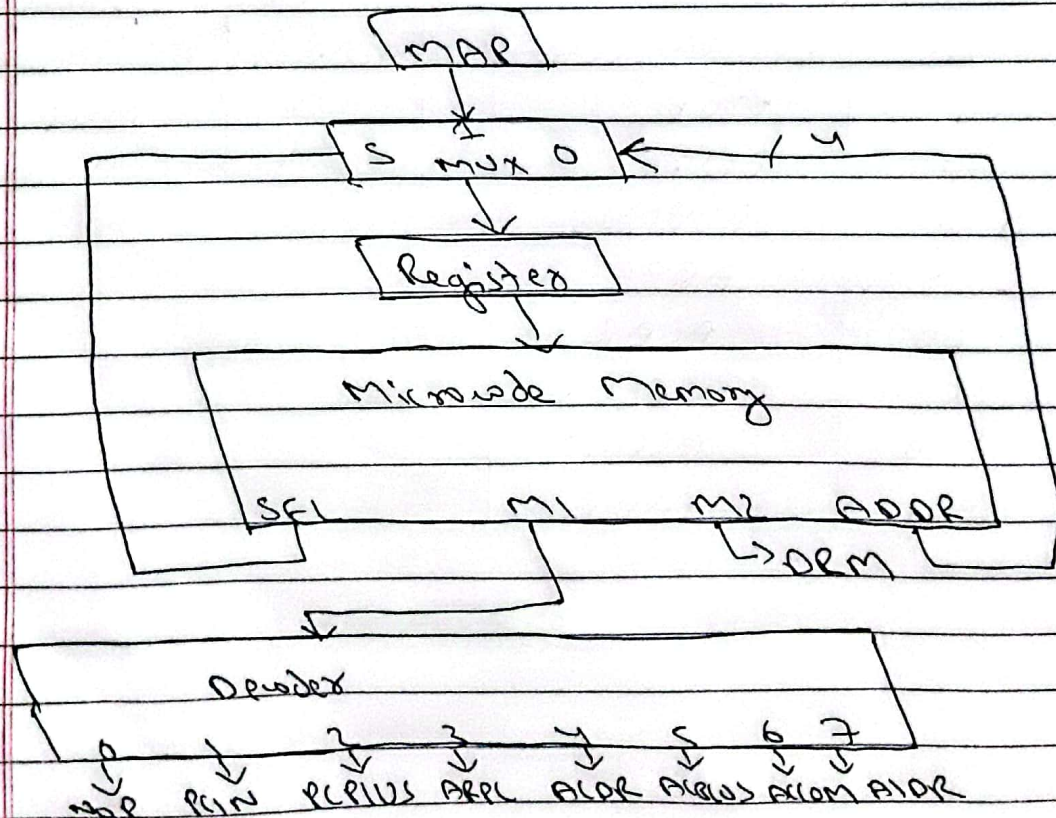
Date: _____

Page: _____

Now, obtain the vertical microcode

State	Address	SEL	M1	M2	ADDR
Fetch1	0000	0	011	0	0001
Fetch2	0001	0	001	1	0010
Fetch3	0010	1	111	0	XXXX
COM1	1000	0	110	0	0000
JREL1	1010	0	010	0	0000
OR1	1100	0	000	1	1101
OR2	1101	0	100	0	0000
SUB11	1110	0	000	1	1111
SUB12	1111	0	101	0	0000

Generating microoperations from vertical microcode.



Chapter 5

~~(Unit asked in)~~
(Mostly numerical asked, sometimes fill)

Q) What is microsequencer.

Ans: A microsequencer is a control unit component in a computer's processor.

- It is responsible for generating the sequence of addresses that fetch microinstruction from memory.
- These microinstructions guide the processor's operations such as data movement, arithmetic and logical operations.
- The microsequencer uses control signals to manage the flow of operations, ensuring that instructions are executed in the correct order.

Example

- Consider a scenario where a computer needs to add two numbers.
- The microsequencer generates addresses to fetch microinstructions from memory.
- These microinstructions direct the processor to fetch the numbers from memory, perform the ~~adding~~ addition using the ALU, and store the result back.
- The microsequencer ensures that each step happens in the correct sequence by generating the appropriate addresses for each microinstruction.

1.2) Explain horizontal and vertical microcode.

a) Horizontal microcode

- Parallelism: It emphasizes simultaneous activation of multiple control signals in a single clock cycle for efficient execution.
- Each ~~cont~~ microinstruction includes multiple control signal.
- It is suited for processor with simple/straight-forward operations that can be executed concurrently.
- It is effective for operations where multiple components need to be manipulated in parallel, such as arithmetic and memory access.

b) Vertical microcode

- Sequential execution: It performs operation step by step.
- Each microinstruction triggers only one control signal.
- It is suitable for processor which need to perform complex task.
- It is effective for operations involving conditional branches, complex instruction set and decision making processes.

Imp

11/3) Differentiate between hardwired and microprogrammed control unit.

Hardwired Control Unit	Microprogrammed Control Unit
1) It is a sequential circuit that generates control signals.	1) It generates the control signals with the help of micro instructions stored in control memory.
2) It consists of counter, decoder & combinational logic.	2) There are horizontal and vertical microcode.
3) It has limited flexibility, as the changes require hardware modifications.	3) It is highly flexible, as the changes can be made by altering micro instructions.
4) It cannot handle complex instructions as the circuit design for it becomes complex.	4) It can handle the complex instruction.
5) The development for design and implementation is less.	5) It takes longer time for design and implementation.
6) It is more faster due to direct hardware control.	6) It is slower than hardwired control unit.
7) It is harder to debug due to fixed hardware.	7) It is easier to debug and test.
8) It is suitable for processors with simple instruction sets.	8) It is suitable for processors with complex instruction sets or frequent updates.